Evaluation of ImageJ and geometry functions

Gabriel Arellano

2026-09-18

Contents

1 Goal and functions to evaluate **2**

2 Evaluation of coordinate recovery code **3**
2.1 How the ImageJ CTFS code and algorithm works . . . 3
2.2 Should we keep the ImageJ workflow? . . . 4
2.3 Alternative algorithms . . . 5
Affine transformation . . . 5
Homography . . . 6
2.4 Practical comparison of algorithms . . . 7
Reference set of points with known location . . . 7
Six paper cases . . . 10
Coordinate recovery . . . 11
Error assessment . . . 12
2.5 Decision . . . 19
Retain workflow, change code . . . 19
Abandon Condit's algorithm . . . 19
Homography > affine . . . 20

3 Evaluation of geometry functions **20**
3.1 Euclidean and line distances . . . 20
3.2 Construction of lines . . . 22
3.3 Line and segment intersections . . . 24
3.4 Drawing helpers and angle bisectors . . . 27
3.5 Rectangle membership and overlap . . . 27
3.6 Circles and ellipses . . . 31
3.7 Cartesian and polar coordinates . . . 33
3.8 Decision . . . 34

4 Due diligence for dependents and dependencies **35**
4.1 Source code 35
4.2 Dependents that could break 38
4.3 Dependencies that will not be needed anymore 40

5 Removal of few auxiliary functions **42**

6 Updated dependency network **42**

7 Closing status **46**

1 Goal and functions to evaluate

The goal is to make the fgeo2 body of code lighter and easier to maintain, while preserving calculations and workflows that remain useful to ForestGEO and the Alliance for Tropical Forest Science. Here I evaluate the ImageJ functions and the geometry functions inherited from CTFS. Most of the code is part of a workflow for converting digitized tree maps into local coordinates, but several of the geometry functions also have uses outside ImageJ. In particular, I evaluate are all 25 functions in `utilities-geometry.r` and all four functions in `topography-imageJ.r`.

This evaluation focuses on:

- identifying functions that can be removed because they add little value or have better alternatives;
- identifying calculations and coordinate conventions worth keeping; and
- checking dependencies, so that removing one function does not leave other code broken.

I evaluate a slightly updated version of `fullplot.imageJ()`: the development of the quadrat-related code removed its dependence on `convert.rowcol()`. That small change avoided breaking the code during the quadrat update, but did not constitute an in-depth evaluation or validation of the image-to-coordinate conversion.

In this document I refer to Richard Condit's old script distribution as "CTFS", the formal R package as `ctfs`, and the later package family as `fgeo`. fgeo2 is the current code development/replacement project, for the moment a script-based distribution.

2 Evaluation of coordinate recovery code

2.1 How the ImageJ CTFS code and algorithm works

The goal of Condit's algorithm is to reduce the error in tree-level locations using the four corners as control points, going from clicked coordinates to adjusted coordinates. In the adjusted coordinate system, the four corners are mapped to the supplied rectangular dimensions. The rest of the coordinates are transformed accordingly.

Condit's *Converting ImageJ Output to Local Plot Coordinates Using R* (2011) explains the original workflow. Maps are scanned, corners and trees are marked in ImageJ, and labelled pixel coordinates are exported as text. `fullplot.imageJ()` reads these files, converts coordinates, combines map sections and prepares a table for database upload. Pages 1–4 describe file names, submaps, checking one folder before processing a whole plot, and checking the resulting coordinate ranges. An HTML version is also available.

`imageJ.to.lxly()` expects columns named `x`, `y` and `label`. Four records identify the corners, in physical order lower-left, upper-left, upper-right and lower-right. Labels are compared without case. `fullplot.imageJ()` defaults to `corners = c("p1", "p2", "p3", "p4")` in that order and passes them explicitly to the converter. A direct call to `imageJ.to.lxly()` instead defaults to `P2`, `P1`, `P4`, `P3`. These differing defaults make it particularly important to match corner labels to the actual maps. The remaining records identify trees. Image pixels can have a top-left origin and downward-positive y; the named corners define the intended local orientation. Incorrect corner identities can therefore reverse or reflect the coordinates.

The code constructs a line for each side of the map. For each tree, `distance.to.side()` calculates three distances to a side: the perpendicular distance and two distances along lines parallel to its adjacent sides. It returns their mean and coefficient of variation. If the mean distances to the left, right, bottom and top sides are d_L, d_R, d_B and d_T, the local coordinates are

$$lx = w\frac{d_L}{d_L + d_R}, \qquad ly = h\frac{d_B}{d_B + d_T},$$

where w and h are the supplied map dimensions. For points inside a correctly labelled rectangle, these ratios give the intended coordinates, including when the rectangle is rotated. For a general quadrilateral, the method is not an explicitly fitted affine or projective transformation. The distances are unsigned: a point outside the map can be assigned coordinates inside it. For example, a point one unit left of a ten-unit-wide rectangle has distances 1 and 11 to its vertical sides; the ratio assigns it `lx = 10 / 12`.

The geometry functions represent lines by intercept and slope, `(b, m)`. For a vertical line, `m` is infinite and `b` is the x coordinate. `SectionCorrection()` adds the width and/or height for four clockwise submap suffixes in a two-by-two arrangement. `fullplot.imageJ()` returns

`tag`, `lx`, `ly` and `quadrat`, with optional text-file output. These are local plot units, normally metres. The function does not calculate plot-global `gx,gy` or geographic coordinates.

The reported `lefterror` and `bottomerror` use coefficients of variation among the three distances, scaled by map dimensions and averaged over trees. They are not control-point residuals or a calibrated measure of registration uncertainty. A zero side distance also makes the coefficient of variation undefined. Missing or duplicate corners, coincident controls, crossed boundaries, non-finite coordinates and ambiguous file matches are not systematically checked. Nor does the returned table guarantee unique tags or alignment with an existing census table.

There is an ambiguity in the 2011 tutorial: it describes four 10 × 10 m submaps within a 20 × 20 m quadrat, but the worked calls use `gridsize = c(20,20)` with four suffixes. `SectionCorrection()` offsets by the supplied grid dimensions, so those literal arguments do not place four 10 × 10 m sections inside a 20 × 20 m quadrat. The dimensions and suffixes need to be checked against the actual maps.

2.2 Should we keep the ImageJ workflow?

[*Note: we call it the "ImageJ workflow" because clicking has traditionally been done in ImageJ software, and that label has been inherited into the R functions. The algorithm itself operates just on numerical coordinates tables in R. ImageJ is not a strict requirement.*]

The workflow addresses a real error-correction step because digitizing hand-drawn paper maps does not produce perfectly registered coordinates. Paper maps, scanning and manual digitization remain practical at some sites. The problematic part is the custom algorithm and complicated geometry helpers used for the correction. Dependency on ImageJ or external software is not strict — and should remain like that. Any practical tool useful to get labelled sets of coordinates should suffice.

The original CTFS workflow returns local coordinates that identify a position within a quadrat. A second step is required to obtain global or plot-level coordinates. As discussed in the evaluation of quadrat-related code, this separation is a point of vulnerability. It is not a major weakness, though.

This evaluation covers 29 functions. Coordinate recovery uses only a subset of them, but its chain of ImageJ and geometry helpers still adds substantial maintenance. A short replacement would make the code easier to understand and maintain.

The problematic or obsolete nature of this code were recognized almost 10 years ago. The earlier function-by-function evaluation, discussed during development of `ctfs` and `fgeo` in 2016-2017, suggested deprecating the ImageJ functions and all the listed geometry functions. That evaluation involved Sean McMahon, Stuart Davies, Mauro Lepore and myself (Gabriel Arellano). The results of that discussion were stored in a table edited directly by McMahon and Arellano; see `evaluation/old-evaluation-table.xlsx`.

However, there are also reasons to preserve the workflow, with or without updates to specific functions. Although the workflow and tutorial are more than 15 years old, there are more recent documented applications using the original CTFS code, conventions and instructions. The 2022 *ForestGEO Database Handbook*, version 03.31.22, page 23 and Appendix E (pages 39–41), still describes this workflow, including the modified PointPicker plugin and the CTFS conversion code. Jiménez, Quintanar and Guevara's *Uso de ImageJ y R para la digitalización de datos de ubicación de árboles en inventarios forestales* (also 2022) demonstrates `fullplot.imageJ()` and reports R 4.2.0. The Harvard Forest hf253 dataset methods (used at least in 2019, possibly later) describe use of ImageJ 1.46 and CTFS code to obtain stem coordinates from scanned census maps. La Planada ForestGEO site, in Colombia, plans to use paper maps during the full census and the click-based digitization process (Stuart J. Davies, personal communication, September 2026).

These sources are not enough to estimate how many sites use this workflow now, but they suggest that it may still be in use at different sites. At least, the old CTFS script-based distribution and the associated tutorials should remain available in easy-to-access repositories. Feasible replacements, more robust or easy to maintain, must be considered.

2.3 Alternative algorithms

The core task of the CTFS code is to implement Condit's algorithm to obtain a best estimate of point location.

The problem is how to transform clicked coordinates into adjusted coordinates. Many source-to-destination transformations can be solved using linear algebra (see this). Two standard algorithms could solve the same problem as Condit's algorithm:

- **affine transformation**, appropriate for flatbed scans; and
- **homography**, or planar projective transformation, appropriate for images with perspective, such as photographs.

Both algorithms can be implemented using linear algebra through applications of `solve()` and `qr.solve()` in base R.

Affine transformation

The affine transformation is theoretically appropriate for flatbed scans and approximately perpendicular photographs, where the map can be corrected with translation, rotation, constant scaling and shear. It requires at least 3 non-collinear control-point pairs. With the usual four corners, all four can be used in a least-squares fit. Additional control points can also be fitted by least squares; for example, if corners of sub-quadrats of known location are represented in the map.

```
apply_affine_transformation <- function(known.xy.of.controls,
                                        clicked.xy.of.controls,
                                        clicked.xy.of.points)
{
    # Inputs are numeric matrices with x and y columns.
    # Both control matrices must use the same row order.

    # Fit the transformation from known to clicked coordinates.
    coefficients <- qr.solve(cbind(1, known.xy.of.controls),
                             clicked.xy.of.controls)

    # Invert the transformation to recover point coordinates in metres.
    estimated.xy.of.points <- cbind(
        clicked.xy.of.points[, 1] - coefficients[1, 1],
        clicked.xy.of.points[, 2] - coefficients[1, 2]
    )
    estimated.xy.of.points <- estimated.xy.of.points %*%
        solve(coefficients[-1, ])

    estimated.xy.of.points
}
```

Homography

The homography, or planar projective transformation, is theoretically appropriate when photographing a flat map introduces appreciable perspective. It requires at least 4 control-point pairs in a non-degenerate arrangement. Additional control points can also be fitted by least squares; for example, if corners of subquadrats of known location are represented in the map.

```
apply_homography_transformation <- function(known.xy.of.controls,
                                            clicked.xy.of.controls,
                                            clicked.xy.of.points)
{
    # Inputs are numeric matrices with x and y columns.
    # Both control matrices must use the same row order.
    x <- clicked.xy.of.controls[, 1]
    y <- clicked.xy.of.controls[, 2]
    gx <- known.xy.of.controls[, 1]
    gy <- known.xy.of.controls[, 2]

    # Each control contributes two equations for the eight coefficients.
```

```
    equations <- rbind(
        cbind(x, y, 1, 0, 0, 0, -gx*x, -gx*y),
        cbind(0, 0, 0, x, y, 1, -gy*x, -gy*y)
    )
    coefficients <- qr.solve(equations, c(gx, gy))

    # Transform the points and convert from homogeneous coordinates.
    homography <- matrix(c(coefficients, 1), nrow = 3, byrow = TRUE)
    estimated.xy.of.points <- cbind(clicked.xy.of.points, 1) %*%
        t(homography)
    estimated.xy.of.points <- estimated.xy.of.points[, 1:2, drop = FALSE] /
        estimated.xy.of.points[, 3]

    estimated.xy.of.points
}
```

2.4 Practical comparison of algorithms

The issues with scanning paper maps are not purely projective. Physical paper can also be folded, stretched or damaged. It is unclear whether Condit's algorithm is more robust to these problems than affine or homography transformations. To compare the three algorithms, I created a grid of known points, printed it, scanned or photographed the paper, clicked the points in the resulting images, and run those points through the error-correction algorithms. The difference between known and recovered locations measures performance under test conditions.

Reference set of points with known location

This R code creates a set of points with known locations and four corners to use as control.

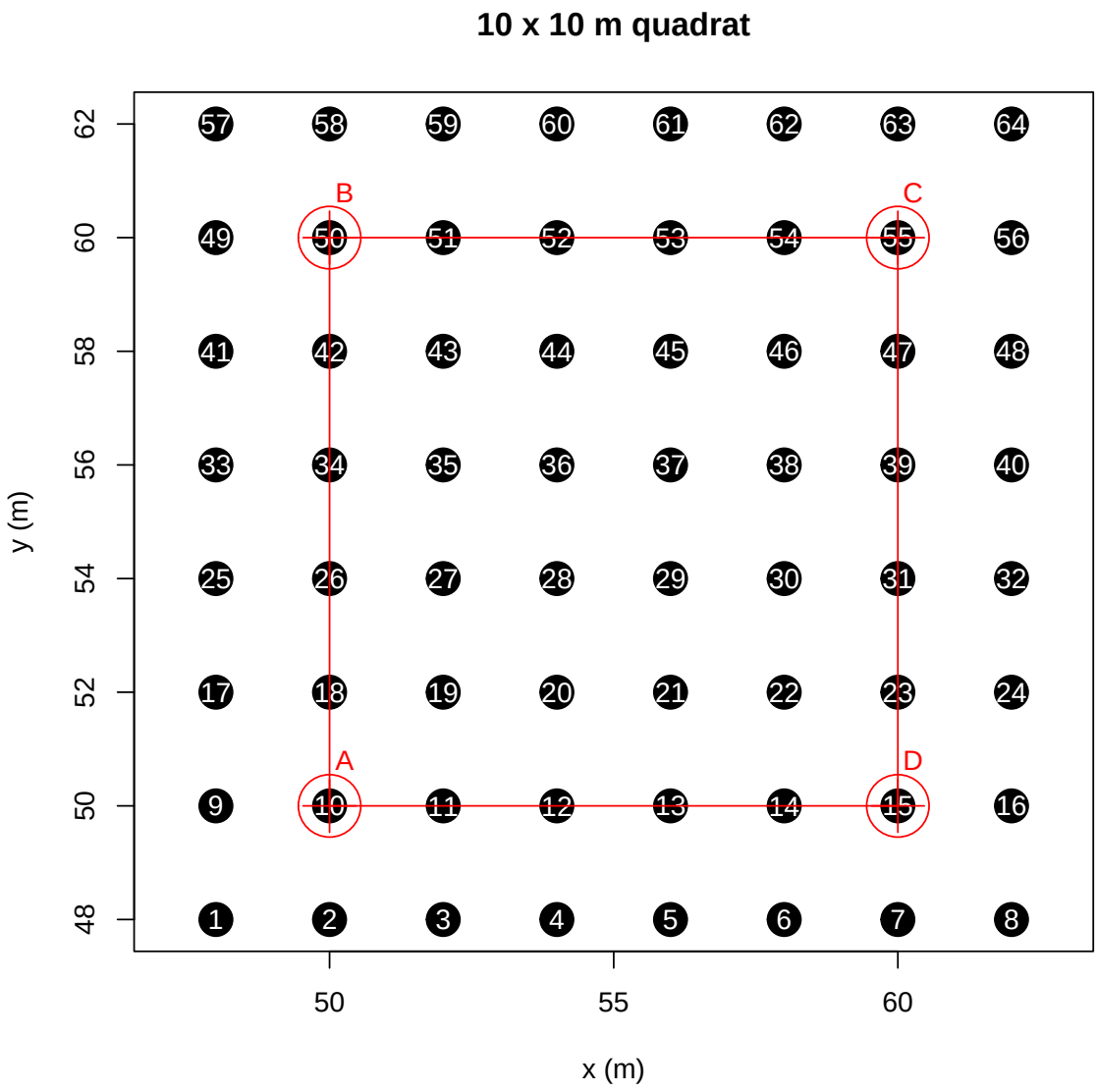
```
# Locate the fgeo2 project.
p <- system2("git", c("rev-parse", "--show-toplevel"), stdout = TRUE)
if(!is.null(attr(p, "status")) || length(p) != 1L)
    stop("Run this evaluation inside the fgeo2 Git repository.")
p <- normalizePath(p)

coordinate.recovery.path <- file.path(
    p, "data_examples", "coordinate_recovery_test"
)

# main parameters
```

```r
l = 10 # side of the quadrat
n = 6 # resolution of the test grid

# corners
x0 = 50
x1 = x0 + l
y0 = 50
y1 = y0 + l

# grid
step = l/(n-1)
xs <- seq(from = x0 - step, to = x1 + step, by = step)
ys <- seq(from = y0 - step, to = y1 + step, by = step)
xy <- expand.grid(x = xs, y = ys)
xy$id <- paste0("p", seq_len(nrow(xy)))

# classify points as inside, edge or outside
xy$where <- "inside" # initialize
xy$where[xy[,"x"] == x0 | xy[,"x"] == x1] <- "edge"
xy$where[xy[,"y"] == y0 | xy[,"y"] == y1] <- "edge"
xy$where[xy[,"x"] < x0 | xy[,"x"] > x1] <- "outside" # overwrites some edges
xy$where[xy[,"y"] < y0 | xy[,"y"] > y1] <- "outside" # overwrites some edges

# plot the grid of points
reference.pdf <- file.path(coordinate.recovery.path,
    "coordinate_recovery_test_grid.pdf")
pdf(reference.pdf)

plot(xy[,c("x", "y")],
     asp = 1,
     cex = 4,
     pch = 20,
     main = "10 x 10 m quadrat",
     xlab = "x (m)",
     ylab = "y (m)")
text(x = xy[,"x"], y = xy[,"y"], labels = 1:nrow(xy), col = "white")

# add the quadrat location
a = step/2.5
points(c(x0, x0, x1, x1), c(y0, y1, y1, y0), cex = 5, pch = 21, col = "red")
points(c(x0, x0, x1, x1), c(y0, y1, y1, y0), cex = 3, pch = 3,  col = "red")
text(c(x0, x0, x1, x1) + a/3, c(y0, y1, y1, y0) + a, c("A", "B", "C", "D"), col = "red")
```

```
lines(c(x0, x0, x1, x1, x0), c(y0, y1, y1, y0, y0), col = "red")

dev.off()
```

```
pdf
  2
```

```
knitr::include_graphics(reference.pdf)
```



Six paper cases

The figure was printed three times, and the prints were treated like this:

- nothing done, taken from the printer as it is;
- crumpled into a tight ball, then flattened; and
- crumpled into a tight ball, flattened, broken into pieces by hand, then reassembled in one try.

These three papers were then:

- scanned with not much care; and
- photographed with a cell phone, by hand, while sitting: not perfectly from the top but not using an absurd unrealistic angle either.

Cases A–C are scans and cases D–F are photographs. Within each group, the first case is the untreated paper, the second is the crumpled paper, and the third is the paper that was broken and reassembled.

```
pdf.paths <- file.path(coordinate.recovery.path,
    paste0("test ", LETTERS[1:6], ".pdf"))

y.limits = list(
    'A' = c(1000, 2700),
    'B' = c(1000, 2700),
    'C' = c(1000, 2700),
    'D' = c(1500, 2500),
    'E' = c(750, 2500),
    'F' = c(1000, 2500)
)

old.par <- par(
    mfrow = c(2, 3),
    mar = c(0, 0, 2, 0)
)

for(i in seq_along(pdf.paths))
{
    case <- LETTERS[i]

    map.image <- magick::image_read(
        pdf.paths[i],
        density = "300x300"
```

```
    )[1]

    map.raster <- as.raster(map.image)

    plot(
        map.raster,
        ylim = y.limits[[case]],
        interpolate = FALSE
    )

    title(
        main = paste("Case", case),
        line = 0.5
    )
}

par(old.par)
```

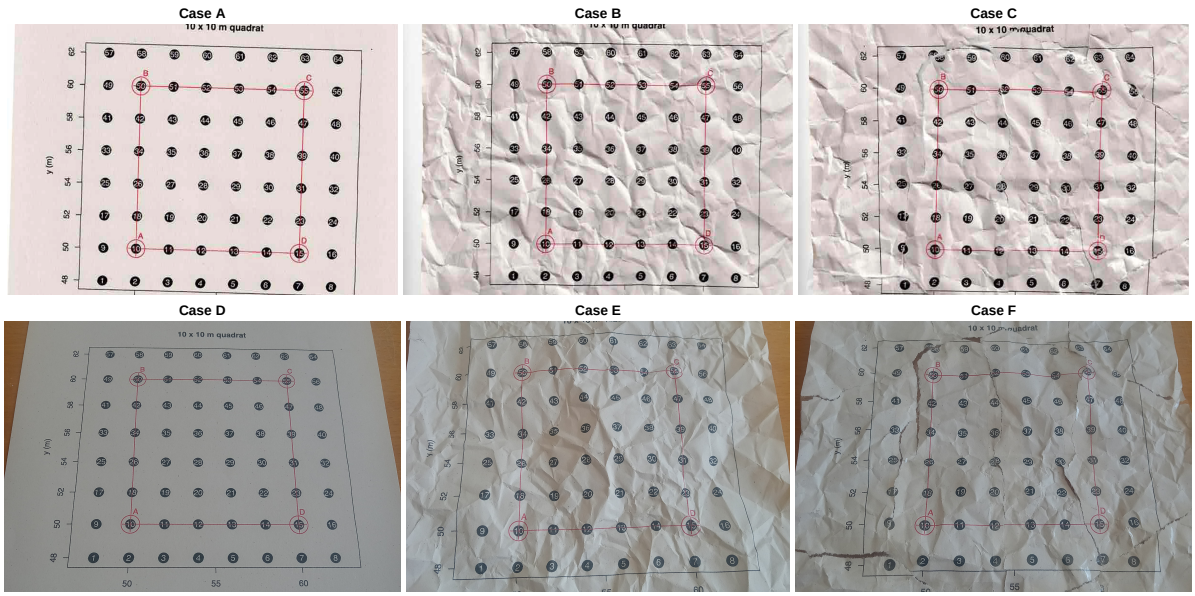


Figure 1: Coordinate-recovery test cases A–F.

Coordinate recovery

I have not used the complete original workflow through ImageJ because the transformation only requires the clicked coordinates. Instead, I used R's `locator()` to click the four corners first,

followed by the known points in numerical order. See `evaluation/coordinate-locator.R` for a simple code. A more complete version may be developed for distribution within fgeo2. The resulting CSV files are stored in `data_examples/coordinate_recovery_test/`. They look like this:

```
example <- read.csv(file.path(coordinate.recovery.path, "points test A.csv"))
head(example, 10)
```

```
         id         x        y
1  corner_A  812.1397 1296.653
2  corner_B  843.7221 2311.597
3  corner_C 1866.1054 2284.412
4  corner_D 1836.6817 1265.737
5        p1  597.3031 1094.112
6        p2  801.7940 1091.630
7        p3 1014.3395 1079.806
8        p4 1212.6647 1077.081
9        p5 1416.9866 1071.030
10       p6 1622.1295 1063.558
```

In all cases, the coordinates are in pixels, internal to each individual image. They are not in the same scale and they do not need to be.

Error assessment

I calculate error as the Euclidean distance between each known point and its recovered location. The four controls are used to fit each transformation and are not included among the validation points. Errors are expressed in centimeters.

The affine and homography transformations use the functions defined above. Condit's algorithm is loaded from the archived CTFS scripts:

```
condit.path <- file.path(p, "evaluation", "Condit")

source(file.path(condit.path, "startCTFS", "startCTFS.r"))
startCTFS(condit.path)

stopifnot(exists("imageJ.to.lxly"),
          exists("distance.to.side"))
```

The archived `distance.to.side()` passes one-column data frames to a numerical distance calculation and therefore fails for ordinary points. The following repair changes only the extraction of the intersection coordinates. The original function remains available as `distance.to.side.original`.

```
distance.to.side.original <- distance.to.side

distance.to.side <- function(pt, target, side1, side2)
{
    perpendicular <- perpendicular.distance(
        b = target[1], m = target[2], x = pt[1], y = pt[2]
    )

    parallel <- drop(as.matrix(parallel.line(
        b = side1[1], m = side1[2], x = pt[1], y = pt[2]
    )))
    intersection <- intersection.of.lines(
        b1 = parallel[1], m1 = parallel[2],
        b2 = target[1], m2 = target[2]
    )
    distance1 <- xydist(pt[1], pt[2], intersection$x, intersection$y)

    parallel <- drop(as.matrix(parallel.line(
        b = side2[1], m = side2[2], x = pt[1], y = pt[2]
    )))
    intersection <- intersection.of.lines(
        b1 = parallel[1], m1 = parallel[2],
        b2 = target[1], m2 = target[2]
    )
    distance2 <- xydist(pt[1], pt[2], intersection$x, intersection$y)

    distances <- c(perpendicular, distance1, distance2)
    c(mean(distances), sd(distances) / mean(distances))
}
```

The following code runs the three algorithms. Identifiers are matched explicitly so the calculation does not depend on row order. Condit's result is converted from coordinates local to the 10×10 m quadrat to the known coordinates of the reference grid.

```
case.names = LETTERS[1:6]
algorithm.names = c("Condit (repaired)", "Affine", "Homography")
```

```
known.controls <- data.frame(
    id = paste0("corner_", LETTERS[1:4]),
    x = c(x0, x0, x1, x1),
    y = c(y0, y1, y1, y0)
)
known.xy.of.controls <- as.matrix(known.controls[, c("x", "y")])
known.xy.of.points <- as.matrix(xy[, c("x", "y")])

error.results <- list()
result.number <- 0L

for(current.case in case.names)
{
    clicked <- read.csv(file.path(
        coordinate.recovery.path,
        paste0("points test ", current.case, ".csv")
    ))

    control.order <- match(known.controls$id, clicked$id)
    point.order <- match(xy$id, clicked$id)
    stopifnot(!anyNA(control.order), !anyNA(point.order))

    clicked.xy.of.controls <- as.matrix(
        clicked[control.order, c("x", "y")]
    )
    clicked.xy.of.points <- as.matrix(
        clicked[point.order, c("x", "y")]
    )

    affine.coordinates <- apply_affine_transformation(
        known.xy.of.controls,
        clicked.xy.of.controls,
        clicked.xy.of.points
    )
    homography.coordinates <- apply_homography_transformation(
        known.xy.of.controls,
        clicked.xy.of.controls,
        clicked.xy.of.points
    )

    # The CTFS function reads label, x and y from a text file.
    condit.input <- clicked[, c("id", "x", "y")]
```

```
    names(condit.input)[1] <- "label"
    condit.file <- tempfile(fileext = ".csv")
    write.csv(condit.input, condit.file, row.names = FALSE)

    condit.result <- imageJ.to.lxly(
        textfile = condit.file,
        delim = ",",
        lowerleft = "corner_A",
        upperleft = "corner_B",
        upperright = "corner_C",
        lowerright = "corner_D",
        gridsize = c(1, 1)
    )$coord
    unlink(condit.file)

    condit.order <- match(xy$id, condit.result$tag)
    stopifnot(!anyNA(condit.order))
    condit.coordinates <- cbind(
        x = condit.result$lx[condit.order] + x0,
        y = condit.result$ly[condit.order] + y0
    )

    estimated.coordinates <- list(
        "Condit (repaired)" = condit.coordinates,
        "Affine" = affine.coordinates,
        "Homography" = homography.coordinates
    )

    for(current.algorithm in algorithm.names)
    {
        estimated <- estimated.coordinates[[current.algorithm]]
        result.number <- result.number + 1L
        error.results[[result.number]] <- data.frame(
            case = current.case,
            algorithm = current.algorithm,
            id = xy$id,
            point.class = xy$where,
            error.cm = 100 * sqrt(
                (estimated[, 1] - known.xy.of.points[, 1])^2 +
                (estimated[, 2] - known.xy.of.points[, 2])^2
            )
        )
```

```
    }
}

error.by.point <- do.call(rbind, error.results)
row.names(error.by.point) <- NULL
```

The table reports the mean error and its standard deviation. The first three rows pool the six cases; the following rows retain the individual test conditions. Each case contains 64 validation points: 16 inside, 20 on the edge and 28 outside the quadrat.

```
format_error <- function(x)
    sprintf("%.0f (%.0f)", mean(x), sd(x))

summary.cases = c("All cases", case.names)
point.groups = c("Overall", "Inside", "Edge", "Outside")

error.table <- expand.grid(
    Algorithm = algorithm.names,
    Case = summary.cases,
    stringsAsFactors = FALSE
)
error.table <- error.table[, c("Case", "Algorithm")]

for(current.group in point.groups)
{
    error.table[[current.group]] <- sapply(seq_len(nrow(error.table)), function(i)
    {
        keep <- error.by.point$algorithm == error.table$Algorithm[i]

        if(error.table$Case[i] != "All cases")
            keep <- keep & error.by.point$case == error.table$Case[i]

        if(current.group != "Overall")
            keep <- keep & error.by.point$point.class == tolower(current.group)

        format_error(error.by.point$error.cm[keep])
    })
}

knitr::kable(
    error.table,
    align = c("l", "l", "r", "r", "r", "r"),
```

```
    caption = "Mean (SD) Euclidean location error, in centimetres. The four fitting controls
)
```

Table 1: Mean (SD) Euclidean location error, in centimetres. The four fitting controls are excluded.

Case	Algorithm	Overall	Inside	Edge	Outside
All cases	Condit (repaired)	165 (179)	14 (12)	8 (8)	364 (51)
All cases	Affine	19 (19)	14 (13)	13 (11)	26 (23)
All cases	Homography	7 (8)	6 (6)	5 (4)	10 (10)
A	Condit (repaired)	161 (184)	3 (1)	3 (1)	364 (51)
A	Affine	3 (2)	3 (1)	3 (1)	4 (1)
A	Homography	3 (2)	3 (1)	3 (1)	4 (2)
B	Condit (repaired)	161 (182)	3 (2)	4 (2)	363 (51)
B	Affine	4 (2)	3 (2)	4 (2)	5 (2)
B	Homography	4 (2)	3 (2)	4 (2)	5 (3)
C	Condit (repaired)	163 (183)	5 (3)	5 (2)	365 (50)
C	Affine	9 (9)	5 (3)	5 (2)	14 (12)
C	Homography	9 (9)	5 (3)	5 (3)	13 (12)
D	Condit (repaired)	169 (176)	26 (5)	12 (12)	363 (51)
D	Affine	33 (14)	26 (5)	23 (10)	43 (13)
D	Homography	3 (2)	2 (1)	2 (1)	4 (2)
E	Condit (repaired)	172 (177)	30 (11)	12 (8)	366 (52)
E	Affine	33 (17)	31 (11)	22 (8)	41 (21)
E	Homography	11 (8)	10 (7)	9 (7)	13 (10)
F	Condit (repaired)	167 (178)	16 (6)	12 (8)	364 (52)
F	Affine	32 (20)	17 (6)	23 (8)	47 (21)
F	Homography	14 (11)	12 (6)	8 (4)	19 (14)

The same results can be inspected as distributions. The logarithmic scale keeps the ordinary errors visible beside the much larger errors produced when Condit's unsigned distances map outside points back toward the quadrat.

```
error.by.point$algorithm <- factor(
    error.by.point$algorithm,
    levels = algorithm.names
)
algorithm.colors = c("grey60", "#E69F00", "#0072B2")

old.par <- par(mfrow = c(1, 3), mar = c(7, 4, 2, 1))
```

```r
for(current.class in c("inside", "edge", "outside"))
{
    current.error <- error.by.point[
        error.by.point$point.class == current.class,
    ]

    boxplot(
        error.cm ~ algorithm,
        data = current.error,
        log = "y",
        col = algorithm.colors,
        main = tools::toTitleCase(current.class),
        ylab = "Error (cm, log scale)",
        las = 2
    )
}

par(old.par)
```

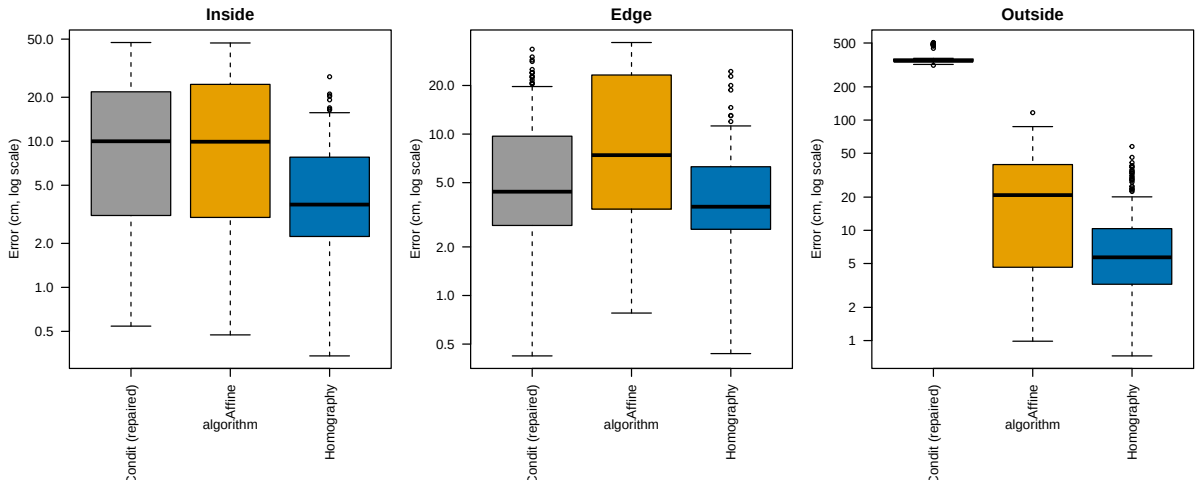


Figure 2: Point-location error by algorithm and location relative to the quadrat, pooled across the six cases. The y axis is logarithmic.

2.5 Decision

Retain workflow, change code

There is a possibility that some teams will use this general workflow, mid-term. A full replacement, including an R point clicker to replace imageJ software / plugins, seems to require minimal stand-alone code. I will give it a go.

The entire workflow is or will be soon obsolete. Clearly, work in tablets is superior to work in paper, then scan, then click, then compile. For that reason, it is not worth to develop extensive documentation or tutorials, neither sophisticated code that tries to maximize automation or easier routines for many maps. I will focus on a minimum viable product. Teams who need more help can contact me.

Abandon Condit's algorithm

Condit's algorithm works well with the "inside" and "edge" points, but it has no place in a light and easy-to-maintain version of the code. Affine and homography transformations do the same, better, for a lower cost:

- Condit's method requires many custom geometry helpers; affine and homography need only short, standard linear-algebra calculations.
- Inconsistent corner defaults: `fullplot.imageJ()` and `imageJ.to.lxly()` assume different corner orders, potentially reflecting the map or creating other practical vulnerabilities.
- Known implementation bug: `distance.to.side()` fails without correcting how intersection coordinates are extracted.
- Incorrect exterior behavior: unsigned distances fold outside points inside, potentially producing duplicate valid-looking records in adjacent quadrats. Affine and homography preserve exterior positions, treating corners as controls, not constraints.
- Corners vs. controls: affine and homography can benefit from any number of control points, including internal quadrats of known location or reference trees with well-validated coordinates from previous censuses. Condit's algorithm can use only four corners, which are treated both as controls and constraints.
- Condit's algorithm returns local coordinates: the local-to-global conversion introduces additional opportunities for errors. The proposed affine / homography implementation returns global coordinates directly, and skips entirely the problematic dichotomy between local and global coordinates. Why this discussed in detail in the evaluation of quadrat-related code).
- Condit's implementation cannot process images from photographs well, performing substantially worse than the homography transformation.

For back-compatibility and as a perk for those who want to still use Condit's algorithm, the tutorial should include a quick fix or warning regarding its known bug. But fgeo2 will not retain Condit's algorithm neither complicate things by trying to retain everything.

Homography > affine

Homography is clearly superior in images with perspective (photographs) and only marginally worse than affine transformation for flatbed scans. The affine transformation does have theoretical properties that makes it better behaved in the [four corners] + [clicking error] case: it uses least squares fit, while homography fits exactly through the clicked (erroneous) corners as if they were total truth. In practice, this is not a serious issue for the homography transformation: added error to tree locations should be small (several cm) even for large clicking errors in corners (1/2 m), relative to the error in the affine transformation. Such inferior performance is of smaller magnitude than field error in mapping stems.

Therefore, the main recommendation should be to use the homography transformation in general, with affine as a plan B when there is a missing corner, or there is unusually large error in clicked corner location. The tutorial can include another small test to confirm this, with the case of more than 4 points of control.

3 Evaluation of geometry functions

Most geometry functions seem part of a complicated sub-network of helpers to imageJ code, and, at first glance, they should be removed as part of the replacement of the old imageJ workflow. The 2016-2017 review of the code recommended complete removal. In addition to this general view, the current evaluation identifies specific problems/bugs, and the specific alternatives in well-maintained R.

3.1 Euclidean and line distances

`xydist()`, `xydistvect()` and `xydistmat()` calculate Euclidean distances between paired Cartesian coordinates. According to the criterion that a small repetition is better than a small dependency, these simple distance functions should be substituted by simple code like `sqrt((x1 - x2)^2 + (y1 - y2)^2)`. When performance is important, there are specific implementations like **`Rfast::Dist()`** or **`parallelDist::parDist()`**. Besides, `xydistvect()` and `xydistmat()` fail with vectors, clearly a potential source of bugs when 1-row matrices are converted into vectors automatically by R:

```
xydistvect(pt1 = c(0, 0), pt2 = c(3, 4))
```

```
Error in `pt1[, 1]`:
! incorrect number of dimensions
```

```
xydistmat(pts = c(0, 0, 3, 4))
```

```
Error in `pts[, 1]`:
! incorrect number of dimensions
```

`perpendicular.distance()` calculates the distance from a point to an infinite line, while `pt.to.segment()` attempts to calculate the shortest distance to a finite line segment and return the nearest point. These functions have unclear or unreliable behavior in important cases.

```
# An extremely steep line produces NaN rather than a distance.
# The true distance from (1, 0) to this near-vertical line
# should be approximately 1.
perpendicular.distance(b = 0, m = 1e308, x = 1, y = 0)
```

```
[1] NaN
```

```
# The point (10, 0) is closest to the endpoint (1, 0).
# The returned distance is 1, but the nearest coordinates are NA.
pt.to.segment(
    x = c(10, 2),
    y = c(0, 0),
    x1 = 0,
    y1 = 0,
    x2 = 1,
    y2 = 0
)
```

```
      x        y    index distance
     NA       NA        2        1
```

For spatial objects, `sf` provides maintained implementations:

```
sf::st_distance(point, segment)
sf::st_nearest_points(point, segment)
```

If a future point-pattern analysis requires explicit projection onto segments, `spatstat.geom::project2segment` is a well-maintained alternative.

Recommendation: Remove `xydist()`, `xydistvect()`, `xydistmat()` and `pt.to.segment()`. Retain `perpendicular.distance()` temporarily because it is used by `minum.perpdist()` and, indirectly, `majoraxisreg()`. Review that regression chain against maintained major-axis implementations such as `smatr::ma()` before removing it.

3.2 Construction of lines

`perpendicular.line()`, `parallel.line()` and `pts.to.interceptslope()` represent lines using slopes and intercepts. This representation becomes awkward for vertical lines and produces inconsistent behavior when scalar and vector inputs are combined.

```
# The perpendicular to a horizontal line through (2, 3)
# should be represented as the vertical line x = 2.
#
# Instead, the function approximates it with a very large
# finite slope and intercept.
perpendicular.line(b = 0, m = 0, x = 2, y = 3)
```

```
              b            m
1 -3.266248e+16 1.633124e+16
```

```
# The first requested parallel line has finite slope.
# The second is vertical and should retain x = 3.
#
# The returned intercept for the second line is incorrect.
parallel.line(
    b = c(0, 0),
    m = c(1, Inf),
    x = c(2, 3),
    y = c(4, 5)
)
```

```
Warning in inter[minf] <- x: number of items to replace is not a multiple of
replacement length
```

```
  b   m
1 2   1
2 2 Inf
```

```
# Coincident points do not define a unique line.
# The function nevertheless returns a vertical line with infinite slope.
l <- pts.to.interceptslope( pt1 = c(1, 1), pt2 = c(1, 1))
l
```

```
  b   m
1 1 Inf
```

```
# This mixed scalar/data-frame input describes two lines.
# The second should be vertical at x = 1, but its intercept is NA.
l <- pts.to.interceptslope(
    pt1 = c(1, 0),
    pt2 = data.frame(
        x = c(2, 1),
        y = c(1, 2)
    )
)

l
```

```
   b   m
1 -1   1
2 NA Inf
```

For a finite slope, the intercept of a parallel line through a known point requires only:

```
m = 1
x = 2
y = 4

b <- y - m * x
b
```

```
[1] 2
```

It seems the treatment of lines in CTFS code is flawed by design – perhaps contaminated by the way linear models are represented. Point-direction or normal-form line representations avoid treating vertical lines as exceptional slopes, and are more general and better representations. General spatial intersections and distances can be delegated to `sf`.

Recommendation: Remove `perpendicular.line()`, `parallel.line()` and `pts.to.interceptslope()`. In the current `R/new` code, every caller of these functions is also marked for removal below. The former modeling dependents of `parallel.line()` and `pts.to.interceptslope()` are no longer present in `R/new`.

3.3 Line and segment intersections

`intersection.of.lines()` calculates intersections between slope-intercept lines. `intersection.line.curve()` attempts to locate intersections between a line and a piecewise curve. `is.between()` tests whether coordinates fall within the bounding ranges of a segment, and `line.intersection.pts()` attempts to construct an intersection from endpoint representations. These functions form a intersection system with inconsistent treatment of vertical, parallel and coincident lines:

```
# A vertical line at x = 2 intersects both horizontal
# lines at x = 2.
#
# The vectorized calculation returns NA for the second x value.
intersection.of.lines(
    b1 = 2,
    m1 = Inf,
    b2 = c(0, 1),
    m2 = c(0, 0)
)
```

```
   x y
1  2 0
2 NA 1
```

```
# Distinct parallel lines have no intersection.
# The result should represent that case explicitly,
# rather than returning Inf and NaN.
intersection.of.lines(
    b1 = 0,
    m1 = 0,
    b2 = 1,
    m2 = 0
)
```

```
    x   y
1 Inf NaN
```

```
# The vertical line x = 1.5 intersects this curve segment
# at approximately (1.5, 0.5).
#
# The calculated x coordinate becomes NA.
intersection.line.curve(
    b = 1.5,
    m = Inf,
    curve = data.frame(
        x = c(0, 1, 2),
        y = c(1, 0, 1)
    )
)
```

```
   x   y
2 NA 0.5
```

```
# The point is inside the coordinate ranges of the segment
# but is not collinear with it.
#
# is.between() therefore tests a bounding box rather than
# actual membership in the segment.
# This behaviour is counter-intuitive and scripts using this function can end being very wro
is.between(
    x = 0.5,
    y = 0.2,
    x1 = 0,
    y1 = 0,
    x2 = 1,
    y2 = 1
)
```

```
[1] TRUE
```

```
# Endpoint ownership is inconsistent between diagonal and
# horizontal segments.
c(
    diagonal = is.between(
        x = 1,
        y = 1,
        x1 = 1,
        y1 = 1,
```

```
        x2 = 0,
        y2 = 0
    ),
    horizontal = is.between(
        x = 1,
        y = 0,
        x1 = 1,
        y1 = 0,
        x2 = 0,
        y2 = 0
    )
)
```

```
  diagonal horizontal
      TRUE      FALSE
```

```
# The function calls the undefined line.intersection().
line.intersection.pts(
    line1 = data.frame(
        x = c(0, 1),
        y = c(0, 1)
    ),
    line2 = data.frame(
        x = c(0, 1),
        y = c(1, 0)
    )
)
```

```
Error in `line.intersection.pts()`:
! unused arguments (line1 = data.frame(x = c(0, 1), y = c(0, 1)), line2 = data.frame(x = c(0, 1), y
```

For a tightly bounded calculation, an intersection can be implemented directly with determinant or vector algebra. For general geometry, use maintained spatial predicates and operations:

```
sf::st_intersection(line1, line2)
sf::st_intersects(line1, line2)
```

Recommendation: Remove `intersection.of.lines()`, `intersection.line.curve()`, `is.between()` and `line.intersection.pts()`. Once the ImageJ workflow is deprecated, none has a remaining external dependency that warrants temporary retention.

3.4 Drawing helpers and angle bisectors

`segmentPt()` draws a segment between two points, while `drawrectangle()` draws a quadrilateral from supplied vertices. These are small wrappers around base graphics, and clearly not needed. Base graphics already provides direct drawing primitives: `segments()`, `polygon()`, `rect()`, …

`angleBisector()` constructs a line intended to bisect the angle defined by three points. The angle-bisector implementation uses complicated slope and quadrant logic and fails a simple hand-checkable example.

```
# These points define a right angle at (1, 0).
#
# The internal bisector should have:
#   slope     = -1
#   intercept =  1
#
# The function instead returns slope 1 and intercept -1.
angleBisector(
    rbind(
        c(0, 0),
        c(1, 0),
        c(1, 1)
    )
)
```

```
 b  m
-1  1
```

A bisector can be calculated more reliably by converting both rays to unit direction vectors and adding them. `atan2()` should be used whenever a quadrant-aware angle is needed.

Recommendation: Remove `segmentPt()`, `drawrectangle()` and `angleBisector()`. They have no unresolved downstream dependencies.

3.5 Rectangle membership and overlap

`insideRectangle()` applies a half-open rectangle-membership convention. `are.ptsinside()` attempts to determine whether points fall inside one rectangle. `ispt.inside()` attempts to summarize the coverage of points by multiple rectangles. `inside.rect()` tests the relationship between two rectangles.

Rectangle membership is ecologically relevant for quadrats and plot maps. However, these general helpers duplicate simple comparisons, contain implementation errors, or overlap with the newer quadrat-assignment code.

```
# This confirms the half-open convention:
# the lower-left boundary is included;
# the upper-right boundary is excluded;
# missing coordinates are treated as outside.
insideRectangle(
    x = c(0, 1, NA),
    y = c(0, 1, 0),
    xrange = c(0, 1),
    yrange = c(0, 1)
)
```

```
[1]  TRUE FALSE FALSE
```

```
# The implementation calls the misspelled and undefined
# function insideRectange().
are.ptsinside(
    pts = data.frame(
        x = 0.5,
        y = 0.5
    ),
    coord = c(0, 0, 1, 1)
)
```

```
Error in `insideRectange()`:
! could not find function "insideRectange"
```

```
# Even after locally correcting the spelling, the documented
# coordinate-vector representation still fails because it is
# indexed as if it were two-dimensional.
local(
    {
        insideRectange <- insideRectangle

        f <- are.ptsinside
        environment(f) <- environment()

        try(
```

```
            f(
                pts = data.frame(
                    x = 0.5,
                    y = 0.5
                ),
                coord = c(0, 0, 1, 1)
            )
        )
    }
)
```

```
Error in coord[, c(1, 3)] : incorrect number of dimensions
```

```
# This substitutes a working are.ptsinside() so that execution
# reaches the next independent defect in ispt.inside().
#
# The function assigns into logical[i] rather than into the
# result object that it is meant to construct.
local(
    {
        are.ptsinside <- function(...) TRUE

        f <- ispt.inside
        environment(f) <- environment()

        try(
            f(
                pt = c(0.5, 0.5),
                areas = data.frame(
                    xlo = 0,
                    ylo = 0,
                    xhi = 1,
                    yhi = 1
                )
            )
        )
    }
)
```

```
Error in f(pt = c(0.5, 0.5), areas = data.frame(xlo = 0, ylo = 0, xhi = 1,  :
  unused argument (areas = data.frame(xlo = 0, ylo = 0, xhi = 1, yhi = 1))
```

```
# The documentation describes containment involving rectangle
# corners, but this example returns TRUE for overlapping bars
# where neither rectangle contains a corner of the other.
inside.rect(
    rect1 = c(0, 4, 1, 2),
    rect2 = c(1, 2, 0, 4)
)
```

```
[1] TRUE
```

```
# Rectangles touching only along a zero-area boundary are
# classified as overlapping.
inside.rect(
    rect1 = c(0, 1, 0, 1),
    rect2 = c(1, 2, 0, 1)
)
```

```
[1] TRUE
```

For one half-open rectangle, the complete calculation is:

```
inside <- x >= xmin &
          x < xmax &
          y >= ymin &
          y < ymax
```

Inclusive rectangle overlap can similarly be expressed directly:

```
overlap <- xmax1 >= xmin2 &
           xmin1 <= xmax2 &
           ymax1 >= ymin2 &
           ymin1 <= ymax2
```

For plot quadrats, `assign_points_to_quadrats()` already represents the boundary convention more explicitly.

Recommendation: Remove `inside.rect()`. Retain `insideRectangle()` temporarily because it is used by `complete.plotmap()`. Retain `are.ptsinside()` temporarily because it is used by `selectrandomquad3()` and `ispt.inside()`. Retain `ispt.inside()` temporarily because it is used by `coverage.diag()`. These functions and their dependents require future review but, for the moment, I do the minimum fixes to solve their current defects (see fgeo2 commit 3a1ed6c).

3.6 Circles and ellipses

`circle()` and `ellipse()` calculate curve coordinates for supplied horizontal coordinates. `fullcircle()` and `fullellipse()` assemble fuller plotting paths from those calculations.

These are plotting helpers rather than ecological geometry primitives. Their output dimensions become inconsistent when requested coordinates fall outside the mathematical curve.

```
# The second x coordinate lies outside a circle of radius 5.
# The function returns a two-element result containing NA.
circle(
    x = c(0, 6),
    radius = 5
)
```

```
[1]  5 NA
```

```
# When all supplied x coordinates lie outside the circle,
# the function returns numeric(0), losing the input alignment.
circle(
    x = c(6, 7),
    radius = 5
)
```

```
numeric(0)
```

```
# The zero-length result propagated from circle() causes
# fullcircle() to fail when constructing its data frame.
fullcircle(
    x = c(6, 7),
    radius = 5
)
```

```
Error in `data.frame()`:
! arguments imply differing number of rows: 5, 3
```

```
# The same alignment problem occurs for an ellipse.
ellipse(
    x = c(0, 8),
    radius = c(7, 5)
)
```

```
[1]  5 NA
```

```
# When all x coordinates fall outside the ellipse,
# the result again has length zero.
ellipse(
    x = c(8, 9),
    radius = c(7, 5)
)
```

```
numeric(0)
```

```
# fullellipse() retains the requested x coordinates but pairs
# them entirely with missing y coordinates.
fullellipse(
    x = c(8, 9),
    radius = c(7, 5)
)
```

```
   x  y
1  8 NA
2  9 NA
3 NA NA
4  8 NA
5  9 NA
```

Circles and ellipses are more naturally generated parametrically:

```
center.x = 0
center.y = 0
radius.x = 7
radius.y = 5

theta <- seq(
    0,
    2 * pi,
    length.out = 361
)

ellipse.coordinates <- data.frame(
    x = center.x + radius.x * cos(theta),
    y = center.y + radius.y * sin(theta)
)
```

Base graphics also provides `graphics::symbols()` for plotting circles, while `sf::st_buffer()` is appropriate when circles represent spatial buffers or neighborhoods rather than merely drawn curves.

Recommendation: Remove `circle()`, `fullcircle()`, `ellipse()` and `fullellipse()`. They have no current dependencies, and maintained or elementary alternatives cover their possible uses more clearly.

3.7 Cartesian and polar coordinates

`cartesian.to.polar()` converts Cartesian coordinates to radius and angle. `polar.to.cartesian()` performs the inverse conversion.

The inverse calculation is elementary and works for ordinary inputs. The forward calculation derives the angle from a one-argument inverse tangent and therefore loses quadrant information. The CTFS implementation is not entirely correct:

```
# The point (-1, 1) is in the second quadrant.
#
# Its quadrant-aware angle should be 3 * pi / 4.
# The function instead returns -pi / 4.
cartesian.to.polar(
    x = -1,
    y = 1
)
```

```
         r      theta
1 1.414214 -0.7853982
```

```
# The incorrect angle means that converting back does not
# recover the original Cartesian coordinates.
polar.coordinates <- cartesian.to.polar(
    x = -1,
    y = 1
)

polar.to.cartesian(
    r = polar.coordinates$r,
    theta = polar.coordinates$theta
)
```

```
  x  y
1 1 -1
```

```
# The origin has a radius of zero, but no uniquely defined
# angular direction. The current calculation returns NaN
# without documenting an origin convention.
cartesian.to.polar(
    x = 0,
    y = 0
)
```

```
  r theta
1 0   NaN
```

The correct quadrant-aware base-R calculations are:

```
x = -1
y = 1

r <- sqrt(x^2 + y^2)
theta <- atan2(y, x)
```

The inverse calculation is similarly direct:

```
x <- r * cos(theta)
y <- r * sin(theta)
```

Recommendation: Remove `cartesian.to.polar()` and `polar.to.cartesian()`. The current forward conversion is incorrect across quadrants, and both operations are clearer as direct base-R expressions.

3.8 Decision

None of the 25 functions reviewed should be maintained permanently as a standalone fgeo2 geometry utility. This conclusion is the same that the one we took in 2016-2017 during the development of `ctfs` and `fgeo` from CTFS code. Four functions, however, still have dependents and can be left for later. Thus:

- Temporarily retain four functions: `perpendicular.distance()`, `insideRectangle()`, `are.ptsinside()` and `ispt.inside()`. They will be reviewed again when reviewing the regression, mapping, random-quadrat and other code that use them.
- Remove the other 21 functions: `xydist()`, `xydistvect()`, `xydistmat()`, `pt.to.segment()`, `perpendicular.line()`, `parallel.line()`, `intersection.of.lines()`, `intersection.line.curve()`, `is.between()`, `line.intersection.pts()`, `pts.to.interceptslope()`, `segmentPt()`, `drawrectangle()`, `angleBisector()`, `inside.rect()`, `circle()`, `fullcircle()`, `ellipse()`, `fullellipse()`, `cartesian.to.polar()` and `polar.to.cartesian()`.

If standard and frequent ecological work requires it, fgeo2 could add geometry again. If that happens, the criteria should be using direct base R for elementary calculations within other functions (small repetition > small dependency) and `sf` for substantive spatial geometry.

4 Due diligence for dependents and dependencies

4.1 Source code

The dependency analysis uses commit `e44f9fd`, dated 18 September 2026. This was the latest commit when the analysis started. The code below reads all scripts under `R/new` at that commit, so later edits do not change the result. Scripts under `R/old` are not included.

```
library(igraph, quietly = TRUE)
```

```
Warning: package 'igraph' was built under R version 4.4.3
```

```

Attaching package: 'igraph'
```

```
The following object is masked _by_ '.GlobalEnv':

    neighbors
```

```
The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

```r
path <- system2("git", c("rev-parse", "--show-toplevel"), stdout = TRUE)
if(!is.null(attr(path, "status")) || length(path) != 1L)
    stop("Run this evaluation inside the fgeo2 Git repository.")
path <- normalizePath(path)

source(file.path(path, "evaluation/0-code-to-source.r"))

evaluation.commit <- "e44f9fd273f1557ef934a2cb7f614312bff212ea"
evaluation.date <- as.Date("2026-09-18")

modules <- c(
    "01_utilities", "02_biomass", "03_spatial",
    "04_abundance_diversity", "05_demography_growth",
    "06_topography_habitat", "07_mapping_plotting"
)

geometry.functions.to.remove <- c(
    "xydist",
    "xydistvect",
    "xydistmat",
    "pt.to.segment",
    "perpendicular.line",
    "parallel.line",
    "intersection.of.lines",
    "intersection.line.curve",
    "is.between",
    "line.intersection.pts",
    "pts.to.interceptslope",
    "segmentPt",
    "drawrectangle",
    "angleBisector",
    "inside.rect",
    "circle",
    "fullcircle",
    "ellipse",
    "fullellipse",
    "cartesian.to.polar",
    "polar.to.cartesian"
)

imageJ.functions.to.remove <- c(
    "fullplot.imageJ",
```

```
    "SectionCorrection",
    "imageJ.to.lxly",
    "distance.to.side"
)

functions.to.remove <- c(
    geometry.functions.to.remove,
    imageJ.functions.to.remove
)

stopifnot(
    length(geometry.functions.to.remove) == 21L,
    length(imageJ.functions.to.remove) == 4L,
    length(functions.to.remove) == 25L,
    !anyDuplicated(functions.to.remove)
)

all.files <- system2("git",
                     c("-C", shQuote(path), "ls-tree", "-r",
                       "--name-only", evaluation.commit),
                     stdout = TRUE)
if(!is.null(attr(all.files, "status")))
    stop("The evaluation commit is not available in this clone.")

module.pattern <- paste(modules, collapse = "|")
scripts <- sort(all.files[grepl(paste0("^(", module.pattern,
                                       ")/R/new/.*\\.[Rr]$"), all.files)])

code.env <- new.env(parent = .GlobalEnv)
definition.rows <- list()

for(file in scripts)
{
    code <- system2("git",
                    c("-C", shQuote(path), "show",
                      paste0(evaluation.commit, ":", file)),
                    stdout = TRUE)
    if(!is.null(attr(code, "status"))) stop("Cannot read ", file)

    # Record each script's definitions using the shared helper.
    connection <- textConnection(code)
    script.functions <- functions_in_script(connection)
```

```
    close(connection)

    eval(parse(text = paste(code, collapse = "\n")), envir = code.env)
    definition.rows[[file]] <- data.frame(Function = script.functions,
                                          Script = file)
}

definitions <- do.call(rbind, definition.rows)
if(anyDuplicated(definitions$Function))
    stop("Some function names are defined in more than one script.")

objects <- mget(ls(code.env, all.names = TRUE),
                envir = code.env, inherits = FALSE)
all.functions <- names(objects)[vapply(objects, is.function, logical(1))]

missing.functions <- setdiff(functions.to.remove, all.functions)
if(length(missing.functions))
    stop("Functions not found: ", paste(missing.functions, collapse = ", "))

stopifnot(setequal(all.functions, definitions$Function))

edges <- get_all_dependencies(envir = code.env)
edges <- unique(edges[, c("from_function", "to_function"), drop = FALSE])
names(edges) <- c("from", "to")
```

There are **21 scripts, 201 functions and 149 directed dependencies** in this snapshot. An arrow goes from a dependency to a function that uses it. The helper includes functions passed to `apply()` and `optim()`, which matter here for `distance.to.side()`, `ispt.inside()` and `minum.perpdist()`.

This is the internal fgeo2 network. Historical copies and functions in external packages are not mixed into it. Undefined calls, such as `line.intersection()`, cannot appear as defined nodes. Calls constructed dynamically and uses in other people's scripts may also be absent.

4.2 Dependents that could break

For each of the 25 functions, the following code identifies direct dependents and follows all dependency levels. Only functions outside the removal set are reported.

```
dependent.rows <- list()
direct.outside <- character()
```

```
outside <- character()

for(function.name in functions.to.remove)
{
    direct <- setdiff(
        edges$to[edges$from == function.name],
        functions.to.remove
    )
    all.dependents <- setdiff(
        downstream_dependents(function.name, edges),
        functions.to.remove
    )
    direct.outside <- union(direct.outside, direct)
    outside <- union(outside, all.dependents)

    if(length(all.dependents) == 0L) next

    dependent.rows[[function.name]] <- data.frame(
        `Function to remove` = function.name,
        `Direct dependents outside removal set` = paste(sort(direct), collapse = ", "),
        `Dependents outside removal set at any level` = paste(
            sort(all.dependents),
            collapse = ", "
        ),
        check.names = FALSE)
}

direct.outside <- sort(direct.outside)
outside <- sort(outside)

if(length(dependent.rows) == 0L)
{
    cat("**Result:** No function outside the removal set depends on any ",
        "of the 25 functions.\n", sep = "")
} else {
    dependent.table <- do.call(rbind, dependent.rows)
    dependent.latex <- knitr::kable(
        dependent.table,
        format = "latex",
        booktabs = TRUE,
        longtable = TRUE,
        row.names = FALSE
```

```
    )
    dependent.latex <- sub(
        "\\begin{longtable}{lll}",
        "\\begin{longtable}{@{}L{4cm}L{9cm}L{9cm}@{}}",
        dependent.latex,
        fixed = TRUE
    )
    cat(
        "\\begin{landscape}\n",
        "\\footnotesize\n",
        "\\setlength{\\tabcolsep}{4pt}\n",
        dependent.latex,
        "\n\\end{landscape}\n",
        sep = ""
    )
}
```

Result: No function outside the removal set depends on any of the 25 functions.

There are **0 direct dependents and 0 dependents at any level outside the removal set**. Removing these 25 functions therefore does not break another function in `R/new` at this snapshot.

4.3 Dependencies that will not be needed anymore

The following check starts with functions called directly by the 25 functions marked for removal. It then retains only functions with no caller left after that removal.

```
upstream.functions <- sort(setdiff(
    unique(edges$from[edges$to %in% functions.to.remove]),
    functions.to.remove
))

remaining.edges <- edges[
    !edges$to %in% functions.to.remove,
    ,
    drop = FALSE
]

uncalled.upstream <- upstream.functions[
    !upstream.functions %in% unique(remaining.edges$from)
]
```

```
orphan.assessment <- c(
    IfElse = paste(
        "is a scalar wrapper around base R `if`.",
        "It contains no ecological calculation and has no remaining caller."
    ),
    logical.grep = paste(
        "recreates `grepl(needle, haystack, fixed = TRUE)`.",
        "It contains no ecological calculation and its only caller was",
        "`fullplot.imageJ()`."
    )
)

stopifnot(setequal(uncalled.upstream, names(orphan.assessment)))

additional.functions.to.remove <- sort(uncalled.upstream)

for(function.name in additional.functions.to.remove)
{
    script <- definitions$Script[
        match(function.name, definitions$Function)
    ]

    cat(
        "-   `", function.name, "()` (`", basename(script), "`) ",
        orphan.assessment[[function.name]],
        " **Decision: remove.**\n",
        sep = ""
    )
}
```

- `IfElse()` (`utilities-utilities.r`) is a scalar wrapper around base R `if`. It contains no ecological calculation and has no remaining caller. **Decision: remove.**
- `logical.grep()` (`utilities-utilities.r`) recreates `grepl(needle, haystack, fixed = TRUE)`. It contains no ecological calculation and its only caller was `fullplot.imageJ()`. **Decision: remove.**

This is a targeted check of dependencies connected to the 25 functions. It does not identify unrelated isolated functions or disconnected groups; those require a separate review.

5 Removal of few auxiliary functions

The evaluation above suggests to remove `IfElse()` and `logical.grep()` functions. I do it directly. In addition, I remove also `drp()` and `pst()` functions, which are small helpers that increase connectivity within the code. Removal of these functions improves the modularity of the code and will make it easier to maintain, hopefully:

- `logical.grep()` has been substituted by `grepl(needle, haystack, fixed = TRUE) %in% TRUE`; the "%in% TRUE" necessary to handly the NAs in the same way as the original.
- `drp()` has been substituted by `drop(as.matrix(x))`, exact equivalent.
- `pst()` has been substituted by paste0(), exact equivalent.
- `IfElse()` was called in three instances of the type `IfElse(length(a) > length(b), length(a), length(b))`, so it has been substituted by `max(length(a), length(b))` instead of some logical construct through base `ifelse()`.

6 Updated dependency network

This figure shows all connected functions remaining under `R/new` at commit `29f4f7d`, after the removals described in this evaluation. Isolated functions are omitted. Different weakly connected subnetworks have different colors; arrows point from a dependency to its caller. The graph describes dependency structure and is not, by itself, an unused-code audit.

```
post.work.commit <- "29f4f7d4a69000d19e25283550231ccd4e269355"
post.work.date <- as.Date("2026-09-18")

post.all.files <- system2(
    "git",
    c("-C", shQuote(path), "ls-tree", "-r", "--name-only",
      post.work.commit),
    stdout = TRUE
)
if(!is.null(attr(post.all.files, "status")))
    stop("The post-work commit is not available in this clone.")

post.scripts <- sort(post.all.files[
    grepl(paste0("^(", module.pattern, ")/R/new/.*\\.[Rr]$"),
          post.all.files)
])
```

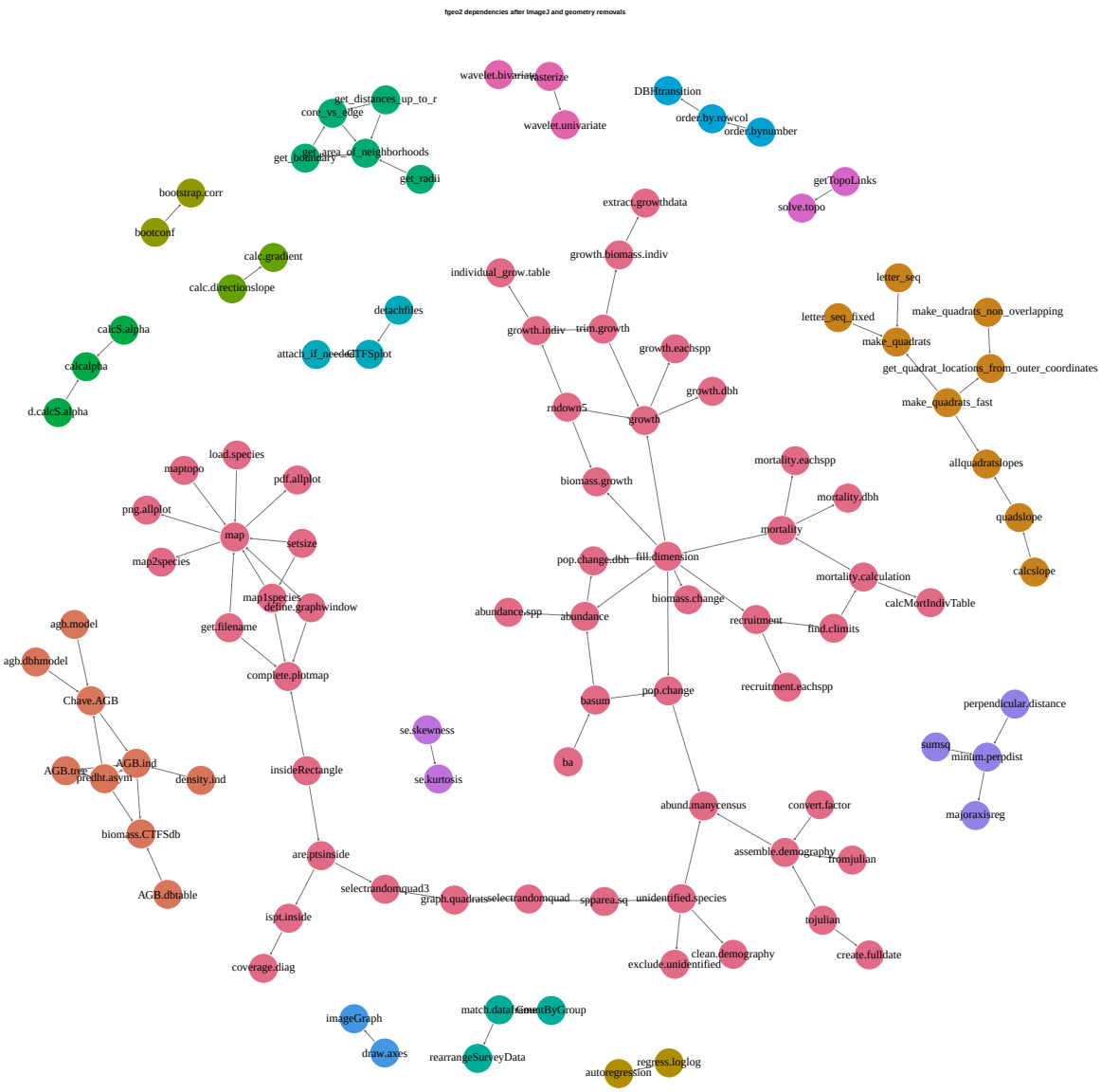
```
post.code.env <- new.env(parent = .GlobalEnv)

for(file in post.scripts)
{
    code <- system2(
        "git",
        c("-C", shQuote(path), "show",
          paste0(post.work.commit, ":", file)),
        stdout = TRUE
    )
    if(!is.null(attr(code, "status"))) stop("Cannot read ", file)

    eval(parse(text = paste(code, collapse = "\n")),
         envir = post.code.env)
}

post.objects <- mget(
    ls(post.code.env, all.names = TRUE),
    envir = post.code.env,
    inherits = FALSE
)
post.functions <- names(post.objects)[
    vapply(post.objects, is.function, logical(1))
]

post.edges <- get_all_dependencies(envir = post.code.env)
post.edges <- unique(
    post.edges[, c("from_function", "to_function"), drop = FALSE]
)
names(post.edges) <- c("from", "to")

removed.after.work <- unique(c(
    functions.to.remove,
    additional.functions.to.remove,
    "drp", "pst"
))
temporarily.retained <- c(
    "perpendicular.distance", "insideRectangle",
    "are.ptsinside", "ispt.inside"
)

post.calls <- unique(unlist(
```

```
    lapply(
        post.objects[post.functions],
        function(fun) codetools::findGlobals(
            fun,
            merge = FALSE
        )$functions
    ),
    use.names = FALSE
))

# Stop if the post-work snapshot no longer represents the closing state.
stopifnot(
    length(post.scripts) == 20L,
    length(post.functions) == 172L,
    !anyDuplicated(post.functions),
    !any(removed.after.work %in% post.functions),
    length(intersect(removed.after.work, post.calls)) == 0L,
    all(temporarily.retained %in% post.functions)
)

# Construct the graph from edges so isolated functions are omitted.
g.after <- graph_from_data_frame(post.edges, directed = TRUE)
subnet <- components(g.after, mode = "weak")
V(g.after)$color <- hcl.colors(
    subnet$no,
    "Dark 3"
)[subnet$membership]

post.work.pdf <- file.path(
    path,
    "evaluation/results",
    paste0(
        "fgeo2-dependencies-", substr(post.work.commit, 1, 7), "-",
        format(post.work.date, "%Y-%m-%d"), ".pdf"
    )
)
dir.create(dirname(post.work.pdf), recursive = TRUE, showWarnings = FALSE)

set.seed(1)

pdf(post.work.pdf, height = 33, width = 33)
par(mar = c(0, 0, 2, 0))
```

```
plot(
    g.after,
    layout = layout.fruchterman.reingold,
    vertex.label = V(g.after)$name,
    vertex.size = 6,
    vertex.color = V(g.after)$color,
    vertex.frame.color = "white",
    vertex.label.color = "black",
    vertex.label.cex = 2,
    edge.color = "gray45",
    edge.width = 0.7,
    edge.arrow.size = 0.3
)

title("fgeo2 dependencies after ImageJ and geometry removals")
dev.off()
```

```
pdf
  2
```

```
post.work.pdf.relative <- file.path(
    "results",
    basename(post.work.pdf)
)
knitr::include_graphics(post.work.pdf.relative)
```



The figure is saved in `evaluation/results/`, with the post-work commit and date in its filename.

7 Closing status

- The 21 deprecated geometry functions and four ImageJ functions were removed from `R/new`; the empty `topography-imageJ.r` script was deleted.

- `perpendicular.distance()`, `insideRectangle()`, `are.ptsinside()` and `ispt.inside()` remain temporarily because they have active callers: the regression chain through `minum.perpdist()` and `majoraxisreg()`, mapping through `complete.plotmap()`, random-quadrat placement through `selectrandomquad3()`, and coverage assessment through `coverage.diag()`. Their status and caller-specific review notes are recorded in `evaluation/function_inventory.xlsx`.
- `drp()`, `pst()`, `IfElse()` and `logical.grep()` were removed after replacing every active call with the documented equivalent expression.
- The replacement coordinate-recovery functions are in `06_topography_habitat/R/new/georeference.R`, with the reproducible workflow in `06_topography_habitat/tutorials/coordinate-recovery.qmd`.